MIT 8103 Advanced Database Systems: Portfolio Summary

Adejoke Ogunyinka (30150302), Master of Information Technology, MIVA Open University
November 2026
GitHub repository: https://github.com/AdejokeOgunyinka-MIVA/Ogunyinka_Adejoke_30150302_MIT-8103_1st-semester-project

1. Selected organisation: Cartify

Cartify is a simulated Nigerian e-commerce platform selling electronics, fashion, groceries, home and beauty products, priced in naira. Its database serves customers (browse, order, pay, track, review), operations staff (products, stock, deliveries), finance staff (payments, refunds) and management (reports). The major data are customers and addresses, categories, products with price and stock, orders and line items, payments, deliveries and reviews. The key operations are adding products, placing an order while reducing stock atomically, recording payments, tracking deliveries, searching the catalogue, viewing order history and reporting top products and revenue by state. The same case study is used throughout all five portfolios.

2. Portfolio 1: Database Design and Modelling

I designed a nine-table relational schema in PostgreSQL 17: customers, addresses, categories (self-referencing for sub-categories), products, orders, order_items, payments, deliveries and reviews. Two many-to-many relationships (orders to products, customers to products) are resolved through junction tables with unique constraints on the foreign-key pair. Every table uses a surrogate integer primary key; all relationships are enforced with foreign keys; money is `NUMERIC(12,2)`; status columns, prices, stock and ratings are guarded by CHECK constraints. The schema is in third normal form, with two documented denormalisations: `order_items.unit_price` records the price actually paid, and `orders.total_amount` is set inside the ordering transaction for reporting speed. A constraint-test script proves the database rejects duplicate emails, negative stock, invalid statuses, orphan foreign keys and out-of-range ratings. A generator script produced 5,000 customers, 500 products, 50,000 orders and about 150,000 order lines with Nigerian names and cities.

Evidence: `portfolio-1-database-design/` (ER diagram in Mermaid and dbdiagram.io PNG, `01_schema.sql`, constraint test output, row-count screenshots).

3. Portfolio 2: Query Processing and Optimisation

Six business queries (order history, catalogue search, category listing, shipped orders by date, top products, revenue by state) were run with `EXPLAIN (ANALYZE, BUFFERS)` before any custom index existed; all six were sequential scans. I then created seven indexes, each justified by a query: composite indexes with the equality column before the range or sort column, indexes on the foreign-key join columns that PostgreSQL does not create automatically, and an expression index on `lower(name)` for case-insensitive prefix search. The customer order-history query dropped from 887 pages read to 16 and from 2.6 ms to 0.08 ms, with the sort step eliminated; the date-range query improved nine-fold. Two queries did not improve, correctly: both aggregate most of their tables, so a sequential scan is cheaper than index lookups. For the top-products report I used a materialised view with its own index, reducing 2,898 pages to 10. The analysis also covers the write cost of indexes and the need to rewrite the search query to match the expression index.

Evidence: `portfolio-2-query-optimisation/results/` (before and after plans, index sizes), README comparison table.

4. Portfolio 3: Transactions and Concurrency

A single-session script demonstrates the place-order transaction: a successful order committed with stock reduced and total computed; an order that fails on an out-of-stock item and is fully rolled back (order count unchanged); and a SAVEPOINT that drops only the unavailable item while committing the rest. Paired-session scripts, interleaved deterministically with `pg_sleep`, demonstrate a non-repeatable read under

READ COMMITTED and a stable snapshot under REPEATABLE READ. The oversold-stock race was reproduced: with one unit in stock and two customers using the read-then-write pattern, both sales succeeded and no constraint fired. I fixed it two ways: pessimistically with `SELECT ... FOR UPDATE` (the second buyer waits, then sees zero and refuses) and optimistically with SERIALIZABLE isolation (the second writer is aborted with a serialization error). A second race, a lost update on price between two staff members, was reproduced and fixed the same way. I also replayed the row-lock race by hand in two terminals.

Evidence: `portfolio-3-transactions-concurrency/results/` (eight output files), two-terminal screenshots.

5. Portfolio 4: NoSQL and Advanced Data Models

I re-modelled the catalogue and orders as MongoDB 7 documents and built them directly from the PostgreSQL data using `json_build_object`, so every embedding decision corresponds to a join that disappears. Product documents embed the category, a per-category `attributes` object (phones carry RAM, storage and colour; rice carries weight and origin), a precomputed rating summary and a bounded array of the three most recent reviews. Order documents embed a customer snapshot, the shipping address, line items, payment and delivery. A `$jsonSchema` validator enforces required fields and value ranges while leaving `attributes` open. Twelve queries show attribute filtering, one-read product pages, text search, multikey indexing on the items array, aggregation pipelines with `$unwind`, a bounded `$push` with `$slice`, and an atomic guarded stock decrement. The suitability analysis argues for polyglot persistence: MongoDB for the catalogue and read models, PostgreSQL for checkout and finance where multi-row atomicity matters.

Evidence: `portfolio-4-nosql/` (export SQL, load and query scripts, data files, outputs, document screenshots).

6. Portfolio 5: Distributed and Cloud Database Exercise

Using Docker Compose I ran a PostgreSQL 17 primary and a hot-standby replica created with `pg_basebackup -R`. A nine-step script loads Cartify into the primary and proves: the replica is streaming (`pg_stat_replication`), a product inserted on the primary is readable on the replica with 0.6 ms replay lag, the replica refuses an UPDATE, a burst of 1,000 orders replicates within a second, and `pg_ctl promote` turns the replica into a writable standalone primary (manual failover). A partitioning script splits 50,000 orders by region as sharding in miniature: a query on the shard key touches one partition, a query without it scatters across all four, and the uneven split (58 % in the South West) illustrates the hot-shard problem and why a hashed customer key would be better. The README discusses synchronous versus asynchronous replication as the CAP trade-off, read-your-writes anomalies, distributed transactions across shards, and how the setup maps to managed cloud services and Nigerian data-residency considerations.

Evidence: `portfolio-5-distributed-cloud/` (compose files, demo output, partition plans, Docker and terminal screenshots).

7. Individual technical reflection

Design choices. I chose e-commerce because every portfolio has a natural example in it: variable product attributes for the document model, the oversold-stock race for concurrency, and read-heavy browsing for replication. I kept surrogate keys, enforced every rule I could in the database rather than the application, and treated the two denormalised columns as recorded historical facts rather than violations.

Challenges. The seed generator initially gave every order to one customer because `random()` inside an uncorrelated subquery is evaluated once; I found it by checking distinct counts. dbdiagram.io rejected the nullable self-referencing foreign key on categories; I kept the design correct rather than bend it to the tool. The replica container failed on data-directory permissions until the cloned directory was set to owner-only. The hardest conceptual point was that a race condition produces no error at all: every statement succeeded and the data was still wrong.

Lessons learned. Read page counts, not milliseconds, when comparing plans. An index that the planner ignores is not a failure; it is information. `FOR UPDATE` changes the meaning of a read. “Schemaless” is a myth; the schema moves. The shard key must be chosen for load distribution before query convenience.

Most of all, reproducibility: writing the interleavings and demos as scripts meant every claim in this document can be re-run.

8. AI Use Declaration

Tool used: Claude Code (Anthropic), model Claude Fable 5.1, used in the terminal throughout the project.

What it was used for: explaining concepts before each portfolio (normalisation, execution plans, isolation levels, document modelling, replication and sharding); drafting SQL, shell and JavaScript scripts, Docker configuration, and first drafts of the README files and this summary; diagnosing errors (the seed-data randomness bug, the dbdiagram validation error, the replica permissions error).

How the output was reviewed and verified: I ran every script myself and compared the output with what the drafts claimed; I read each README in full and rewrote the reflection sections in my own words; I checked every number in the comparison tables against the captured results files; I replayed the row-lock race by hand in two terminals; I took all screenshots personally. I chose the case study, the tools, the schedule, and the design decisions after discussing the trade-offs. A running log of each use and its verification is in `AI-USE-LOG.md` in the repository. I take full responsibility for the accuracy and understanding of everything submitted.

9. References

- PostgreSQL Global Development Group (2025). *PostgreSQL 17 Documentation*: Chapter 11 Indexes; Chapter 13 Concurrency Control; Chapter 27 High Availability, Load Balancing, and Replication; Section 5.12 Table Partitioning. https://www.postgresql.org/docs/17/
- MongoDB, Inc. (2025). *MongoDB Manual 7.0*: Data Modeling; Schema Validation; Aggregation Pipeline; Indexes. https://www.mongodb.com/docs/manual/
- Docker, Inc. (2025). *Docker Compose documentation*. https://docs.docker.com/compose/
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly. Chapters 5 (Replication), 6 (Partitioning), 7 (Transactions).
- Elmasri, R. and Navathe, S. B. (2016). *Fundamentals of Database Systems*, 7th ed. Pearson. Chapters on normalisation and transaction processing.
- Holywell, S. *dbdiagram.io* and *DBML* documentation. https://dbml.dbdiagram.io/
- Decomplexify (2022). *Learn Database Normalization*. YouTube. Nasser, H. *PostgreSQL indexing* and *Transaction Isolation Levels*. YouTube.